

UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBELIARD

Projet IF2B — P26

Démineur

- 1 Andre Jérémie
- 2 Christmann Maxence
- 3 Maurois Romain

Table des matières :

1. Introduction	3
1.1. Contexte et objectifs	3
1.2. Fonctionnalités demandées	3
2. Conception	5
2.1. Architecture du projet	5
2.1.1. main.c : Boucle principale du jeu	5
2.1.2. demineur.h/c : Affichage et logique de jeu	6
2.1.3. initialisation.h/c : Configuration de la partie	6
2.2. Choix techniques	7
3. Test et validation	8
3.1. Test du programme	8
3.2. Corrections Apportées	8
4. Conclusion	10

1. Introduction

1.1. Contexte et objectifs

Ce projet a pour but de mettre en pratique les concepts de programmation en C appris en cours, notamment :

- La gestion des structures de données (tableaux 2D, structures).
- La programmation modulaire (division du code en plusieurs fichiers .c et .h).
- La manipulation de fichiers (pour la sauvegarde et la reprise de partie).
- L'interaction avec l'utilisateur (menus, entrées/sorties).

Ce projet a également permis de développer un jeu complet en respectant un cahier des charges, tout en ajoutant des fonctionnalités avancées comme les bonus/malus.

1.2. Fonctionnalités demandées

- Fonctionnalités de base :

Le jeu devait inclure les éléments suivants :

- Un menu principal avec trois options : Démarrer une partie, Reprendre une partie, Quitter.
- Le choix du mode de jeu (Classique ou Bonus/Malus).
- La configuration de la grille (taille entre 6×6 et 12×12).
- Le choix de la difficulté (Facile, Moyen, Difficile), déterminant le nombre de mines :

Facile : $0.2 \times N^2$ mines.

Moyen : $0.25 \times N^2$ mines.

Difficile : $0.3 \times N^2$ mines.

- Déroulement du jeu:
 - Affichage d'une grille initialement vide (représentée par des points).
 - Révélation des cases via leurs coordonnées (x, y) :
 - Si la case contient une mine, le joueur perd.
 - Si la case est vide (0 mine adjacente), révélation récursive des cases voisines.
 - Si la case contient un nombre (1-8), ce nombre est affiché.
 - Gestion des drapeaux : Possibilité de placer (D) ou retirer un drapeau sur une case.
- Fin de partie:
 - Victoire : Toutes les cases non minées sont révélées.
 - Défaite : Une mine est révélée.
 - Score : Temps écoulé affiché en « minutes : secondes ».
- Mode Bonus/Malus

En plus des règles classiques, le mode Bonus/Malus ajoutait :

 - Cases Brouillage :
 - Quand activées, masquent toutes les cases non révélées pendant 2 tours (affichage de « ? »).
 - Cases +1 UP :
 - Donnent une vie supplémentaire au joueur et le protègent contre la prochaine explosion de mine.
 - Les effets 1 UP sont cumulables.
 - Nombre de bonus/malus :
 - $0.5 \times N$ cases de chaque type (Brouillage et +1 UP) sont placées aléatoirement, sans chevaucher les mines.

- Sauvegarde et reprise :

- Possibilité de sauvegarder la partie à tout moment dans un fichier sauvegarde.csv.

- Sauvegarde de tous les états :

- Taille de la grille, nombre de mines, mode de jeu.
- État des grilles (mines, grille utilisateur, objets).
- Nombre de vies, tours de brouillage, temps écoulé.

- Reprise :

- Rechargement de la dernière partie sauvegardée depuis le menu principal.

2. Conception

2.1. Architecture du projet

Notre projet Démineur est structuré en 4 modules principaux. Cette séparation modulaire a été choisie pour garantir une bonne lisibilité et compréhension du code. Voici l'arborescence du projet :

```
Demineur/  
├─ CMakeLists.txt      # Configuration pour la compilation  
├─ main.c              # Boucle principale du jeu  
├─ demineur.h/c        # Affichage et logique de jeu  
├─ initialisation.h/c  # Configuration de la partie  
└─ sauvegarde.h/c      # Sauvegarde et chargement
```

2.1.1. main.c : Boucle principale du jeu main.c gère :

- Le menu principal (Démarrer, Reprendre, Quitter).
- La boucle principale du jeu (affichage, entrées utilisateur, traitement des actions).

- L'initialisation des variables (taille de la grille, nombre de mines, mode de jeu, etc.).
- La fin de partie (victoire ou défaite).

Exemple : La fonction `main()` appelle `selectionnerOptionMenu()` pour afficher le menu, puis entre dans une boucle où elle appelle `afficherGrille()`, `entree_utilisateur()`, et traite les actions du joueur.

2.1.2. demineur.h/c : Affichage et logique de jeu

Ce module contient les fonctions liées à :

- L'affichage de la grille (`afficherGrille`) : Affiche la grille en tenant compte du brouillage et des cases révélées.
- La gestion des entrées utilisateur (`entree_utilisateur`) : Récupère l'action du joueur (révéler, drapeau, sauvegarder) et ses coordonnées.
- La révélation des cases (`revelerCase`) : Révèle une case et déclenche la révélation en cascade si la case est vide.
- La gestion des bonus/malus (`appliquerBonusMalus`, `gererExplosionMine`) : Active les effets des bonus/malus et gère les explosions de mines.

Exemple : La fonction `revelerCase` utilise une récursivité pour révéler toutes les cases adjacentes vides.

2.1.3. initialisation.h/c : Configuration de la partie

Ce module gère :

- La configuration des paramètres (`configurationPartie`) : Demande à l'utilisateur de choisir le mode, la taille et la difficulté.
- Le calcul du nombre de mines (`mineDifficulte`) : Utilise les formules du sujet ($0.2 \times N^2$, $0.25 \times N^2$, $0.3 \times N^2$) et arrondit avec `roundf`.
- Le placement des mines (`generePositionMines`) : Place les mines aléatoirement sur la grille, sans chevauchement.
- Le calcul des mines adjacentes (`valeurCaseAutourMine`) : Parcourt la grille pour incrémenter le compteur de mines adjacentes pour chaque case.
- Le placement des bonus/malus (`generePositionBrouillage`, `generePosition1UP`) : Place les cases spéciales aléatoirement, sans chevaucher les mines.

Exemple : La fonction `generePositionMines` génère des positions aléatoires pour les mines et vérifie qu'il n'y a pas de doublons.

2.2. Choix techniques

Nous avons utilisé trois grilles 2D pour représenter l'état du jeu:

- mines (type int) :
Stocke le nombre de mines adjacentes (0 à 8) ou la valeur 9 pour une mine.
Avantages :
Permet des calculs arithmétiques simples.
Facilite la révélation en cascade.
Inconvénients :
Nécessite une conversion pour l'affichage.
- grille_utilisateur (type char) :
Stocke l'état visible de chaque case (' ', 'D', 'V', '1'-'8', 'X').
- objets (type int) :
Stocke les bonus/malus (0 = aucune, 10 = brouillage, 11 = +1 UP).

Nous avons utilisé un compteur (`tours_brouillage`) pour gérer l'effet de brouillage.

. Avantages :

- Simplicité : Un simple décrétement à chaque tour suffit.
- Efficacité : Pas besoin de stocker l'état du brouillage dans chaque case.

. Inconvénients :

- Impossible d'avoir plusieurs effets de brouillage simultanés.

Dans `revelerCase`, nous vérifions si la case est déjà révélée.

. Avantages :

- Évite les révélations en double.
- Prévient les boucles infinies.

Nous avons choisi un fichier CSV (`sauvegarde.csv`).

. Avantages :

- Lisibilité : Peut être ouvert/modifié manuellement.
- Portabilité : Format universel (Excel)

Nous avons réalisé l'intégralité de la documentation des fonctions à l'aide de Doxygen.

3. Test et validation

3.1. Test du programme

Pour être sûr que tout fonctionne dans notre démineur, nous avons réalisé beaucoup de test. Nous avons affiché les valeurs dans la console pour vérifier que toutes les données rentrées par l'utilisateur étaient correctes.

Exemple des tests en console pour l'initialisation :

```
DEBUG: Dans la fonction, difficulte vaut : 1
DEBUG: Dans la fonction, mode vaut : 1
DEBUG: Dans la fonction, dimension vaut : 8
12.800000
Nombre de mines : 13
```

De plus, Nous avons vérifié sur des grilles de tailles différentes (6×6 à 12×12) que l'arrondi avec roundf calculait correctement le nombre exact de mines ($0.2\backslash N^2$, $0.25\backslash N^2$ ou $0.3\backslash N^2$) ainsi que les $0.5\backslash N$ cases de Brouillage et +1 UP.

Nous avons simulé le déclenchement d'une mine avec un compteur de vies supérieur à 0. La fonction gererExplosionMine décrémente correctement la vie sans terminer la partie.

3.2. Corrections Apportées

Pour le chargement d'une partie, nous avons utilisé fgetc(fichier) à la place de fscanf(fichier, "%n") car cela créait une erreur d'affichage du tableau lorsque l'on chargeait une partie.

Exemple : La colonne 1 ne s'affichait pas correctement

	1	2	3	4	5	6	7	8	9	10
1	;									
2	;									
3	;									
4	;									
5	;									
6	;									
7	;									
8	;									
9	;									
10	;									

Initialement, les bonus/malus pouvaient écraser une mine ou se superposer. Nous avons corrigé les fonctions `generePositionBrouillage` et `generePosition1UP` pour qu'elles effectuent un contrôle dans la grille mines avant de valider leurs coordonnées.

4. Conclusion

Le projet Démineur nous a permis de mettre en pratique les concepts de programmation en C appris en cours, tout en développant un jeu complet et fonctionnel. Ce projet a également été l'occasion de travailler en équipe pour concevoir, implémenter, tester et documenter une application en respectant un cahier des charges.

Bien que le projet réponde aux exigences du sujet, plusieurs améliorations pourraient être apportées pour le rendre encore plus complet :

- Utiliser des tableaux dynamiques pour économiser de la mémoire
- Pouvoir cumuler les effets de brouillage
- Utiliser des couleurs pour les drapeaux, les nombres...
- Créer des niveaux prédéfinis, plus durs et plus grands.
- Ajouter un compteur de drapeaux et l'afficher en haut de la grille
- Afficher les règles du jeu
- Ajouter des nouveaux bonus/malus comme un bonus reveleTout : Révèle toute une ligne ou une colonne.
- Sauvegarder le meilleur score dans un fichier